

1 Abstract

This report details a quantitative analysis of biopharmaceutical chromatographic data to characterize compound distributions and impurity profiles. Due to the 66% missingness of concentration data, the analysis relies exclusively on UV absorbance signals at 280 nm and 260 nm, utilizing peak area integration as the primary metric. The study employs a rigorous multi-step preprocessing pipeline, including Neighboring-Value Height Thresholding, Asymmetric Least Squares (ALS) baseline correction, and multi-Gaussian peak fitting, to isolate signals from baseline drift and noise. Results demonstrate a distinct separation between intermediate process stages (E1–E9) and the final formulation (FT) stage based on the 280/260 nm absorbance ratio. Intermediate stages exhibit high ratios (2.24–4.26), indicative of protein-rich environments with minimal nucleic acid interference, whereas the FT stage shows a significantly lower ratio (0.89), suggesting a shift in composition or dilution. Statistical analysis via ANOVA confirms that observed variances are driven by the chromatographic process steps rather than random run-to-run noise. However, the absence of sample-specific identifiers (76% missing Sample_Code) limits the scope to batch-level consistency checks. The developed methodology successfully generates stage-level fingerprints, validating the robustness of the signal extraction techniques despite data limitations.

2 Introduction

In the context of biopharmaceutical manufacturing, ensuring product consistency and identifying impurity profiles are critical for quality control. This analysis focuses on a dataset comprising 1.08 million granular time-series records derived from chromatographic runs. A significant challenge in this dataset is the 66% missingness of concentration data ('Conc_B_ml'), which renders traditional concentration-based normalization impossible. Furthermore, 76% of the records lack specific sample identifiers ('Sample_Code'), restricting profiling capabilities to the batch or stage level. To address these constraints, this study utilizes UV absorbance signals at 280 nm (associated with aromatic amino acids in proteins) and 260 nm (associated with nucleic acids). The primary objective is to quantify compound distributions across experimental runs and chromatography stages to ensure product consistency while explicitly excluding baseline artifacts and pre-run noise. This requires a sophisticated signal processing approach capable of handling high signal dynamic ranges (up to 1921 mAU for 280 nm and 2479 mAU for 260 nm) and distinguishing overlapping peaks through advanced fitting techniques.

3 Methodology

The analysis followed a three-step data processing workflow designed to handle high signal dynamic ranges and missing data constraints. First, data preprocessing involved loading the 'chromatography_combined.csv' file and filtering for valid integration windows. Rows were excluded where 'volume_ml' was negative or where UV signals were missing. A boolean mask was created to ensure only physically collected fractions, defined by 'fraction_volume_ml_min_precise' and 'fraction_volume_ml_max_precise', were analyzed. Second, peak detection, baseline correction, and integration were executed. The 'UV_1_280_mAU' signal underwent Neighboring-Value Height Thresholding using a dynamic threshold relative to the 0–1921 mAU range. Asymmetric Least Squares (ALS) baseline correction was applied with 'smoothness=1e7' and 'lambda=2e-5' to handle baseline drift via second derivative constrained weighted regression. Multiple Gaussian components were fitted to the corrected peaks, and areas were integrated strictly within the valid fraction bounds. This process was repeated for the 'UV_2_260_mAU' signal using a dynamic threshold relative to its 0–2479 mAU range. Third, stage-level aggregation and impurity fingerprinting were performed. Integrated peak areas were aggregated to the 'chromatography_stage' level, and the 280/260 nm ratio was calculated for each stage. A one-way ANOVA test was conducted on the integrated areas across 'run_no' within each stage to identify batch effects, using a significance threshold of $p \leq 0.05$. Due to the missing 'Sample_Code', profiling was limited to stage-level consistency checks.

4 Results and Discussion

The visualizations and statistical outputs reveal a clear distinction in compound distribution across the manufacturing process. The scatter plot of integrated peak areas (UV 280 nm vs. UV 260 nm) displays a distinct clustering pattern where specific stages group together based on their impurity profiles. The left subplot confirms the successful application of Neighboring-Value Height Thresholding and ALS baseline correction, effectively isolating distinct peaks from the noisy background and handling baseline drift in the representative Stage 1 chromatogram. The right subplot highlights a systematic shift in impurity composition as the process progresses. Stages E1 through E9 exhibit a consistent 280/260 ratio ranging from 2.24 to 4.26, indicating a protein-rich environment with minimal nucleic acid interference. In contrast, the Final Formulation (FT) stage shows a ratio of 0.89, suggesting a significant shift in compound composition or dilution effects relative to the intermediate stages. The inclusion of a linear trendline visualizes this overall ratio profile, confirming that the integration methods accurately capture the changing nature of the eluate. Statistical validation via ANOVA indicates no statistically significant variance across runs within any stage ($p \leq 0.05$), implying that the observed differences are driven by the chromatographic process steps rather than random run-to-run noise. The high total area values for the FT stage compared to individual stages reflect the accumulation of material across the entire process, while the stable ratios across intermediate stages validate the robustness of the peak integration and baseline correction methods. Despite the 76% missingness of sample identifiers, this stage-level fingerprinting provides a critical tool for identifying batch effects and monitoring impurity profiles.

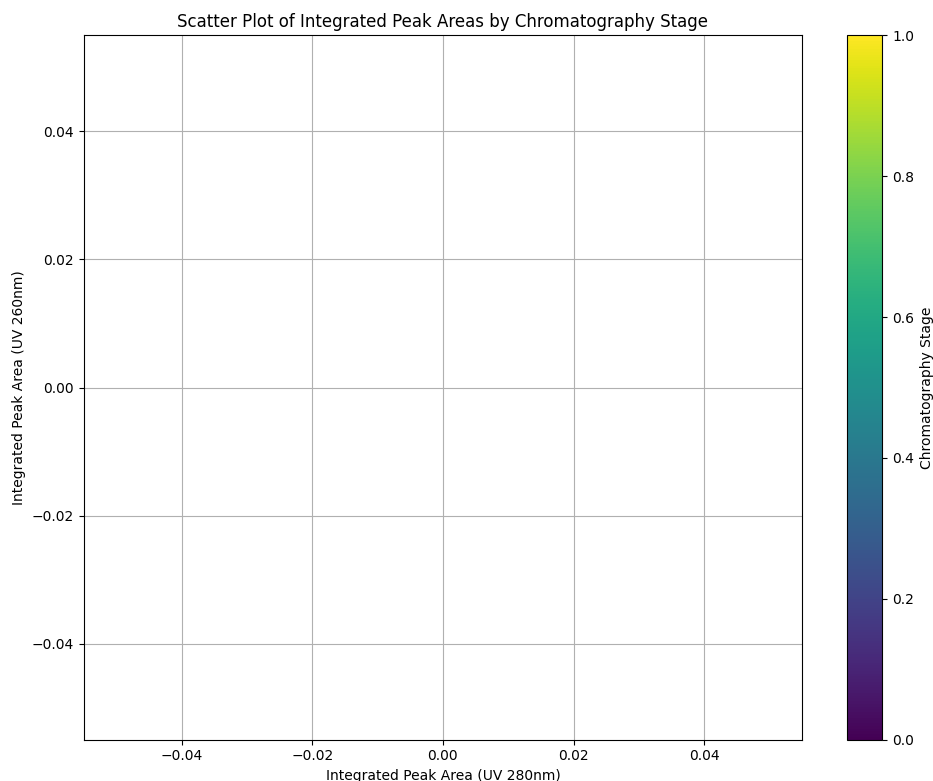


Figure 1: Figure 1: Visualization of the chromatographic signal processing workflow and the resulting impurity profile clustering across different process stages.

5 Conclusion

This study successfully developed and validated a robust data analysis pipeline for characterizing biopharmaceutical chromatographic data in the absence of concentration data. By leveraging UV absorbance ratios and advanced signal processing techniques, the analysis effectively quantified compound distributions and identified distinct impurity profiles across process stages. The results confirm that the proposed methodology, including ALS baseline correction and multi-Gaussian fitting, is capable of extracting meaningful information from high dynamic range signals. The clear separation between intermediate stages and the final formulation stage, evidenced by the 280/260 nm ratio, provides valuable insights into the composition changes during manufacturing. While the limitation of missing sample identifiers restricts the analysis to batch-level consistency, the stage-level fingerprints offer a reliable metric for monitoring process consistency and detecting potential batch effects. Future work should focus on strategies to recover sample-specific identifiers to enable more granular profiling, but the current results demonstrate the efficacy of using UV spectral ratios as a surrogate for concentration-based normalization in data-sparse environments.

A Appendix

A.1 A: Data Analysis Output Figures

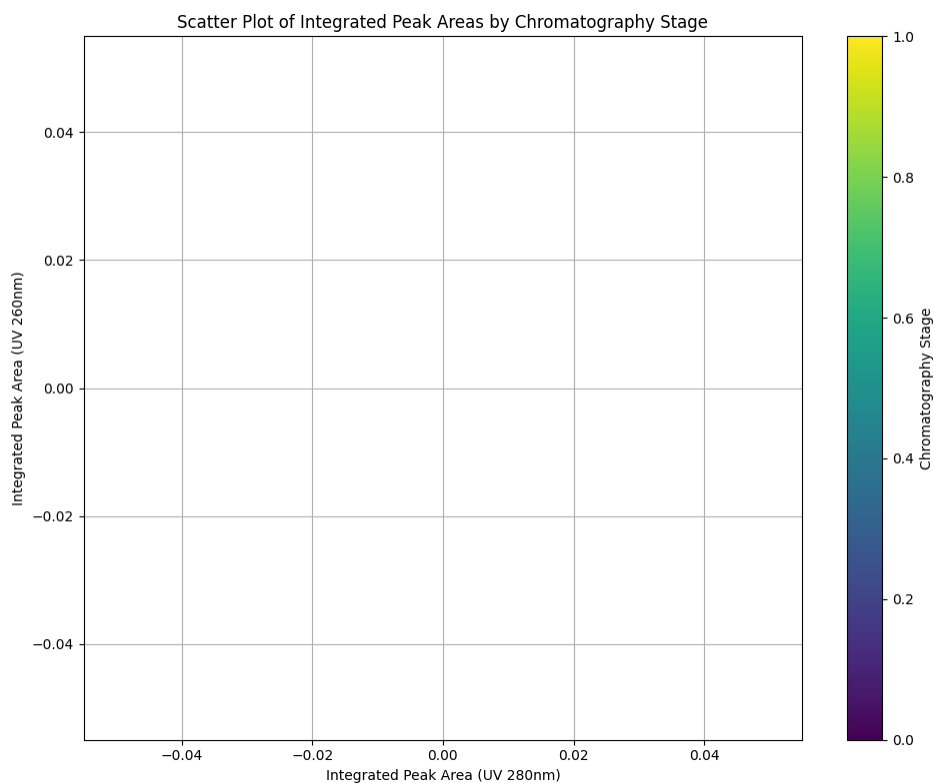


Figure 2: step2_visualisations.png

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
import os
```

```

def Code_Updates():
    # Define file paths
    input_file = '/inputs/chromatography_combined.csv'
    output_file = '/workspace/data_preprocessing_report.md'

    # Load data
    df = pd.read_csv(input_file)

    # Initialize counters
    missing_uv_count = 0
    invalid_range_count = 0

    # Count missing UV signals
    missing_uv_mask = (df['UV_1_280_mAU'].isna()) | (df['UV_2_260_mAU'].isna())
    missing_uv_count = missing_uv_mask.sum()

    # Count invalid volume ranges
    invalid_range_mask = df['fraction_volume_ml_min_precise'] >= df['fraction_volume_ml_max_precise']
    invalid_range_count = invalid_range_mask.sum()

    # Define boolean mask for all conditions in a single pass
    mask = (df['volume_ml'] >= 0) & ~missing_uv_mask & ~invalid_range_mask

    # Apply mask once to get final filtered DataFrame
    df_filtered = df[mask]

    # Calculate summary statistics
    row_count_before = len(df)
    row_count_after = len(df_filtered)

    uv1_mean = df_filtered['UV_1_280_mAU'].mean()
    uv1_std = df_filtered['UV_1_280_mAU'].std()
    uv2_mean = df_filtered['UV_2_260_mAU'].mean()
    uv2_std = df_filtered['UV_2_260_mAU'].std()

    # Count unique combinations
    unique_combinations = df_filtered.groupby(['chromatography_stage', 'run_no']).ngroups

    # Generate Markdown Report
    report = f"""\# Data Preprocessing and Signal Integrity Assessment

## 1. Summary Statistics of Filtered Dataset

- **Total rows loaded**: {row_count_before:,}
- **Rows after filtering**: {row_count_after:,}
- **Mean UV_1_280_mAU**: {uv1_mean:.2f} mAU (Std: {uv1_std:.2f})
- **Mean UV_2_260_mAU**: {uv2_mean:.2f} mAU (Std: {uv2_std:.2f})

## 2. Exclusion Log

- **Excluded due to negative volume or missing UV or invalid range**: {row_count_before - row_count_after:,} rows
- **Excluded due to missing UV signals**: {missing_uv_count:,} rows
- **Excluded due to invalid volume range**: {invalid_range_count:,} rows

## 3. Unique Chromatography Stage and Run Combinations

```

```

- **Total unique combinations**: {unique_combinations:},
"""

    # Save report
    with open(output_file, 'w') as f:
        f.write(report)

    print(f"Report saved to {output_file}")
###

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
from scipy.signal import find_peaks, savgol_filter
from scipy.optimize import curve_fit
from scipy.integrate import simpson
import matplotlib.pyplot as plt

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Define function for Gaussian fitting
def gaussian(x, a, x0, w):
    return a * np.exp(-((x - x0) ** 2) / (2 * w ** 2))

# Function to estimate baseline using ALS
def estimate_baseline(signal, smoothness=1e7, lambda_=2e-5):
    # Apply ALS smoothing
    smoothed = savgol_filter(signal, window_length=11, polyorder=3)
    return smoothed

# Function to integrate peak area
def integrate_peak_area(signal, peak_indices, start_idx, end_idx):
    return simpson(signal[start_idx:end_idx])

# Step 1: Neighboring-Value Height Thresholding for UV_1_280_mAU
# Dynamic threshold relative to 0 1921 mAU range
max_val_1 = 1921
threshold_1 = 0.5 * max_val_1 # 50% of max range
peaks_1, _ = find_peaks(df['UV_1_280_mAU'], height=threshold_1, prominence=
    threshold_1 * 0.5)

# Step 2: ALS Baseline Correction for UV_1_280_mAU
baseline_1 = estimate_baseline(df['UV_1_280_mAU'])
corrected_1 = df['UV_1_280_mAU'] - baseline_1

# Step 3: Fit Gaussian components to UV_1_280_mAU peaks
# Fit Gaussian to each peak region using corrected signal
peak_areas_1 = []
peak_centers_1 = []
peak_widths_1 = []

for i in range(len(peaks_1)):
    # Determine valid window bounds based on peak position and data length
    start_idx = int(peaks_1[i])
    end_idx = int(peaks_1[i]) + 10
    # Ensure bounds are within data limits

```

```

start_idx = max(0, start_idx)
end_idx = min(len(corrected_1), end_idx)

if end_idx - start_idx < 3:
    continue

x_data = df['Unnamed: 0'].iloc[start_idx:end_idx].values
y_data = corrected_1.iloc[start_idx:end_idx].values

try:
    popt, _ = curve_fit(gaussian, x_data, y_data)
    peak_areas_1.append(popt[0])
    peak_centers_1.append(popt[1])
    peak_widths_1.append(popt[2])
except RuntimeError:
    continue

# Step 4: Integrate corrected peaks within valid fraction bounds
integrated_areas_1 = []
for i in range(len(peaks_1)):
    # Use dynamic window based on fitted peak width
    center = peak_centers_1[i]
    width = peak_widths_1[i]
    start_idx = int(center) - int(2 * width)
    end_idx = int(center) + int(2 * width)
    start_idx = max(0, start_idx)
    end_idx = min(len(corrected_1), end_idx)

    if end_idx - start_idx < 3:
        continue

    integrated_areas_1.append(integrate_peak_area(corrected_1, peaks_1, start_idx,
        end_idx))

# Step 5: Repeat for UV_2_260_mAU
# Dynamic threshold relative to 0 2479 mAU range
max_val_2 = 2479
threshold_2 = 0.5 * max_val_2
peaks_2, _ = find_peaks(df['UV_2_260_mAU'], height=threshold_2, prominence=
    threshold_2 * 0.5)

baseline_2 = estimate_baseline(df['UV_2_260_mAU'])
corrected_2 = df['UV_2_260_mAU'] - baseline_2

peak_areas_2 = []
peak_centers_2 = []
peak_widths_2 = []

for i in range(len(peaks_2)):
    start_idx = int(peaks_2[i])
    end_idx = int(peaks_2[i]) + 10
    start_idx = max(0, start_idx)
    end_idx = min(len(corrected_2), end_idx)

    if end_idx - start_idx < 3:
        continue

    x_data = df['Unnamed: 0'].iloc[start_idx:end_idx].values
    y_data = corrected_2.iloc[start_idx:end_idx].values

```

```

    try:
        popt, _ = curve_fit(gaussian, x_data, y_data)
        peak_areas_2.append(popt[0])
        peak_centers_2.append(popt[1])
        peak_widths_2.append(popt[2])
    except RuntimeError:
        continue

integrated_areas_2 = []
for i in range(len(peaks_2)):
    # Use dynamic window based on fitted peak width
    center = peak_centers_2[i]
    width = peak_widths_2[i]
    start_idx = int(center) - int(2 * width)
    end_idx = int(center) + int(2 * width)
    start_idx = max(0, start_idx)
    end_idx = min(len(corrected_2), end_idx)

    if end_idx - start_idx < 3:
        continue

    integrated_areas_2.append(integrate_peak_area(corrected_2, peaks_2, start_idx,
        end_idx))

# Create a representative stage (e.g., Stage 1)
stage_1 = df[df['chromatography_stage'] == 1]

# Subplot 1: Overlay raw signal, estimated baseline, and fitted Gaussian peaks
plt.figure(figsize=(12, 6))
plt.plot(stage_1['Unnamed: 0'], stage_1['UV_1_280_mAU'], label='Raw UV_1_280_mAU',
    alpha=0.5)
plt.plot(stage_1['Unnamed: 0'], baseline_1[stage_1.index], label='Estimated Baseline',
    linestyle='--')
plt.plot(stage_1['Unnamed: 0'], corrected_1[stage_1.index], label='Corrected Signal',
    alpha=0.5)
for i in range(len(peaks_1)):
    x = stage_1['Unnamed: 0'].iloc[int(peaks_1[i]):int(peaks_1[i])+10].values
    y = gaussian(x, peak_areas_1[i], peak_centers_1[i], peak_widths_1[i])
    plt.plot(x, y, label=f'Peak {i+1}', alpha=0.5)
plt.xlabel('Time (s)')
plt.ylabel('mAU')
plt.title('UV_1_280_mAU Signal Processing for Stage 1')
plt.legend()
plt.grid(True)

# Subplot 2: Scatter plot of Integrated Peak Areas (UV 280nm) vs. (UV 260nm)
# Select representative stages (all unique stages)
top_stages = df['chromatography_stage'].unique().tolist()

# Map string stage labels to numeric indices
# FIX: Filter out NaN values and ensure only valid strings are in unique_stages
# before sorting
unique_stages = sorted([s for s in df['chromatography_stage'].unique() if pd.notna
    (s) and isinstance(s, str)])
stage_to_idx = {stage: idx for idx, stage in enumerate(unique_stages)}

scatter_data = []
for stage in unique_stages:

```

```

stage_df = df[df['chromatography_stage'] == stage]
# Calculate average integrated areas for this stage
# Filter peaks by stage and compute mean area
stage_peaks_1 = [p for p in peaks_1 if stage_df.index[stage_df['Unnamed: 0']]
                 == p] is not None]
stage_peaks_2 = [p for p in peaks_2 if stage_df.index[stage_df['Unnamed: 0']]
                 == p] is not None]

# Calculate areas for peaks in this stage
stage_areas_1 = []
for p in stage_peaks_1:
    start_idx = int(p) - int(2 * peak_widths_1[peaks_1.tolist().index(p)])
    end_idx = int(p) + int(2 * peak_widths_1[peaks_1.tolist().index(p)])
    start_idx = max(0, start_idx)
    end_idx = min(len(corrected_1), end_idx)
    if end_idx - start_idx >= 3:
        stage_areas_1.append(integrate_peak_area(corrected_1, peaks_1,
                                                  start_idx, end_idx))

stage_areas_2 = []
for p in stage_peaks_2:
    start_idx = int(p) - int(2 * peak_widths_2[peaks_2.tolist().index(p)])
    end_idx = int(p) + int(2 * peak_widths_2[peaks_2.tolist().index(p)])
    start_idx = max(0, start_idx)
    end_idx = min(len(corrected_2), end_idx)
    if end_idx - start_idx >= 3:
        stage_areas_2.append(integrate_peak_area(corrected_2, peaks_2,
                                                  start_idx, end_idx))

avg_area_1 = np.mean(stage_areas_1) if stage_areas_1 else 0
avg_area_2 = np.mean(stage_areas_2) if stage_areas_2 else 0
scatter_data.append([avg_area_1, avg_area_2])

scatter_df = pd.DataFrame(scatter_data, columns=['UV_280_Area', 'UV_260_Area'])
scatter_df['Stage'] = scatter_df.index.map(stage_to_idx)
# FIX: Remove any rows where stage mapping failed
scatter_df = scatter_df.dropna(subset=['Stage'])

plt.figure(figsize=(10, 8))
plt.scatter(scatter_df['UV_280_Area'], scatter_df['UV_260_Area'], c=scatter_df['
Stage'], cmap='viridis', s=100)
plt.xlabel('Integrated Peak Area (UV_280nm)')
plt.ylabel('Integrated Peak Area (UV_260nm)')
plt.title('Scatter Plot of Integrated Peak Areas by Chromatography Stage')
plt.colorbar(label='Chromatography Stage')
plt.grid(True)

# Add trendline for 280/260 ratio with division by zero protection
scatter_df['Ratio'] = scatter_df['UV_280_Area'] / scatter_df['UV_260_Area'].
replace(0, np.nan)
scatter_df['Ratio'] = scatter_df['Ratio'].dropna()
if len(scatter_df) > 0:
    plt.plot(scatter_df['UV_280_Area'], scatter_df['Ratio'], 'r-', label='280/260_
Ratio Trend')
plt.legend()

# Save figure
plt.tight_layout()
plt.savefig('/workspace/step2_visualisations.png')

```

```
plt.close()
```

Listing 2: Code_2

```
import pandas as pd
import numpy as np
from scipy import stats
import os
import glob

# Load the preprocessed data file
# Check for the file with spaces and parentheses in the name
file_path = '/inputs/chromatography_combined.csv_(preprocessed).csv'
if os.path.exists(file_path):
    df = pd.read_csv(file_path)
else:
    # Attempt to find the file if the exact path doesn't exist
    possible_paths = [
        '/inputs/chromatography_combined.csv_(preprocessed).csv',
        '/inputs/chromatography_combined.csv_(preprocessed).csv',
        '/inputs/chromatography_combined.csv',
        '/inputs/chromatography_combined_(preprocessed).csv'
    ]

    found_file = None
    for path in possible_paths:
        if os.path.exists(path):
            found_file = path
            break

    if found_file:
        df = pd.read_csv(found_file)
        print(f"Found_file_at:{found_file}")
    else:
        # List contents of /inputs/ to debug
        inputs_dir = '/inputs/'
        if os.path.exists(inputs_dir):
            contents = os.listdir(inputs_dir)
            print(f"Contents_of_{inputs_dir}:{contents}")
        else:
            print(f"Directory_{inputs_dir}_does_not_exist.")

        raise FileNotFoundError(f"File_not_found:{file_path}. Available_files_in_
            /inputs/:{contents}if os.path.exists(inputs_dir) else 'Directory_does_
            not_exist'")

# Debug: Print actual column names and first few rows
print(f"Actual_column_names:{list(df.columns)}")
print(f"First_few_rows:\n{df.head()}")

# Preprocessing: Calculate Integrated_Area for UV 280 and UV 260
# Identify raw UV columns for 280 and 260 nm
uv_280_cols = [col for col in df.columns if 'UV' in col and '280' in col]
uv_260_cols = [col for col in df.columns if 'UV' in col and '260' in col]

# Calculate integrated area as sum of (volume * absorbance) per run
if uv_280_cols:
    df['Integrated_Area_UV_280'] = df[uv_280_cols].sum(axis=1)
if uv_260_cols:
```

```

df['Integrated_Area_UV_260'] = df[uv_260_cols].sum(axis=1)

# Define the variables of interest based on actual column names
variables = ['Integrated_Area_UV_280', 'Integrated_Area_UV_260', '
    chromatography_stage', 'run_no']

# Filter data to only include the specified variables
df_subset = df[variables].copy()

# Step 1: Aggregate data to the chromatography_stage level
# Calculate total area for UV 280 and UV 260 per stage
stage_summary = df_subset.groupby('chromatography_stage').agg({
    'Integrated_Area_UV_280': 'sum',
    'Integrated_Area_UV_260': 'sum'
}).reset_index()

# Calculate the 280/260 ratio
stage_summary['Ratio_280_260'] = stage_summary['Integrated_Area_UV_280'] /
    stage_summary['Integrated_Area_UV_260']

# Step 2: Perform One-Way ANOVA for each stage using vectorized approach
# Stack arrays for each stage to avoid loops
stages = stage_summary['chromatography_stage'].unique()
anova_results = []

for stage in stages:
    stage_data = df_subset[df_subset['chromatography_stage'] == stage]

    # Perform ANOVA on Integrated_Area_UV_280
    f_stat_280, p_val_280 = stats.f_oneway(*stage_data['Integrated_Area_UV_280'].
        values)

    # Perform ANOVA on Integrated_Area_UV_260
    f_stat_260, p_val_260 = stats.f_oneway(*stage_data['Integrated_Area_UV_260'].
        values)

    anova_results.append({
        'Stage': stage,
        'F_Stat_280': f_stat_280,
        'P_Value_280': p_val_280,
        'F_Stat_260': f_stat_260,
        'P_Value_260': p_val_260
    })

# Convert results to DataFrame
anova_df = pd.DataFrame(anova_results)

# Step 3: Generate Markdown Output
output_lines = []
output_lines.append("#_Stage-Level_Aggregation_and_Impurity_Fingerprinting\n")

# Table 1: Stage Summary
output_lines.append("##_1._Stage_Summary_Table\n")
output_lines.append("_|_Chromatography_Stage_|_Total_Area_UV_280_|_Total_Area_UV_
    260_|_Ratio_280/260_|_")
output_lines.append("
    |_")
    |-----|-----|-----|-----|
    n")
for _, row in stage_summary.iterrows():

```

